

APMA4302 HW4

John Anthony Labbate

05/06/2026

Problem 1

We consider a two-dimensional incompressible flow with velocity field

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}),$$

and define the vorticity as

$$\boldsymbol{\omega} = \nabla \times \mathbf{v}.$$

In 2D (with $\mathbf{v} = (v_x, v_y, 0)$), the vorticity has only a z -component:

$$\boldsymbol{\omega} = \omega \hat{\mathbf{k}}.$$

The system is:

$$\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T = \frac{1}{\text{Re}} \nabla^2 T, \quad (1)$$

$$-\nabla^2 \mathbf{v} + \nabla p = T \hat{\mathbf{j}}, \quad (2)$$

$$\nabla \cdot \mathbf{v} = 0. \quad (3)$$

We take the curl of

$$-\nabla^2 \mathbf{v} + \nabla p = T \hat{\mathbf{j}}.$$

Using the vector identity $\nabla \times (\nabla p) = 0$, we obtain:

$$-\nabla \times (\nabla^2 \mathbf{v}) = \nabla \times (T \hat{\mathbf{j}}).$$

Since derivatives commute for smooth fields, at least in Cartesian coordinates (I would have to more rigorously check for non-Cartesian coordinates I am not sure),

$$\nabla \times (\nabla^2 \mathbf{v}) = \nabla^2 (\nabla \times \mathbf{v}),$$

so the equation becomes:

$$-\nabla^2 (\nabla \times \mathbf{v}) = \nabla \times (T \hat{\mathbf{j}}).$$

Computing:

$$\nabla \times (T \hat{\mathbf{j}}) = \begin{vmatrix} \hat{\mathbf{i}} & \hat{\mathbf{j}} & \hat{\mathbf{k}} \\ \partial_x & \partial_y & \partial_z \\ 0 & T & 0 \end{vmatrix} = -\hat{\mathbf{i}} \partial_z T + \hat{\mathbf{k}} \partial_x T.$$

In 2D, there is no z -dependence, so $\partial_z T = 0$. Hence:

$$\nabla \times (T \hat{\mathbf{j}}) = \partial_x T \hat{\mathbf{k}}.$$

Thus:

$$-\nabla^2 \omega = \partial_x T \hat{\mathbf{k}}.$$

Taking the z -component as ω only has a z -component if $\mathbf{v}$ is 2D:

$$-\nabla^2 \omega = \frac{\partial T}{\partial x}.$$

Now the streamfunction-vorticity relation. We compute:

$$\mathbf{v} = \nabla \times (\psi \hat{\mathbf{k}}).$$

Using the identity:

$$\nabla \times (\psi \hat{\mathbf{k}}) = \nabla \psi \times \hat{\mathbf{k}},$$

we take the curl again:

$$\nabla \times \mathbf{v} = \nabla \times (\nabla \times (\psi \hat{\mathbf{k}})).$$

Using the vector identity:

$$\nabla \times (\nabla \times \mathbf{A}) = \nabla(\nabla \cdot \mathbf{A}) - \nabla^2 \mathbf{A},$$

with $\mathbf{A} = \psi \hat{\mathbf{k}}$, we get:

$$\nabla \times \mathbf{v} = \nabla(\nabla \cdot (\psi \hat{\mathbf{k}})) - \nabla^2(\psi \hat{\mathbf{k}}).$$

In 2D, ψ has no z -dependence, so:

$$\nabla \cdot (\psi \hat{\mathbf{k}}) = \partial_z \psi = 0.$$

Therefore:

$$\nabla \times \mathbf{v} = -\nabla^2 \psi \hat{\mathbf{k}}.$$

Thus,

$$\omega = -\nabla^2 \psi.$$

Problem 2

Part (a)

Lots of information is outputted by the log-view command when running these different options files. We can compare the timing for each run, and it is clear from the outputted diagnostics that the SNES Solve step usually dominates the time it takes for runs. The convergence rates and run times for these three option files are shown in Table 1.

Options file	KSP iters	Convergence rate ρ	Run time (s)
options_file_direct	1	1.41×10^{-12}	3.011
options_file_split_direct	1	1.43×10^{-12}	3.372
options_file_split_mg	4	2.99×10^{-2}	0.965

Table 1: Solver performance on the 513×513 grid. Convergence rate is computed as $\rho = (r_{\text{final}}/r_0)^{1/k}$, where k is the number of KSP iterations.

Part (b)

Using firedrake code instead to tackle same problem as Part (a) of this problem. Initially, the only set of options included in the firedrake "biharm.py" file are 'pc = "monolithic"' and 'pc = "fieldsplit"'. Comparing these two options gives the following results shown in Table 2.

Firedrake case	KSP iters	Convergence rate ρ	Run time (s)
Monolithic (gmres)	1	4.02×10^{-13}	27.09
Fieldsplit (fgmres)	4	2.06×10^{-2}	24.14

Table 2: Firedrake biharmonic runs with `-log-view`. Convergence rate is $\rho = (r_{\text{final}}/r_0)^{1/k}$, where k is the number of KSP iterations.

I edited the code to mimic exactly the three options files from Part (a) of this problem. When I do this, the parameter cases can be compared in Table 3. Much of the timing seems to come from the initialization of firedrake.

Case label	KSP iters	Convergence rate ρ	Run time (s)
direct	1	1.01×10^{-12}	24.99
split_direct	1	1.02×10^{-12}	23.86
split_mg	4	2.06×10^{-2}	24.70

Table 3: firedrake solver comparison mirroring C option labels on the 512×512 mesh. Convergence rate is $\rho = (r_{\text{final}}/r_0)^{1/k}$, where k is the number of KSP iterations.

Part (c)

Let's compare the direct solvers for the C PETSc code and the firedrake equivalent. See Table 4 for the comparison. The firedrake wrappers by far cost the most time, whereas the the PETSc events are much faster (although still slower when called by firedrake).

Rank	PETSc C (options_file_direct)		Firedrake PETSc solver events		Firedrake wrappers	
	Event	Time (s)	Event	Time (s)	Event	Time (s)
1	SNESolve	2.8142	SNESolve	5.6313	firedrake	23.3200
2	PCSetUp	2.4211	PCSetUp	4.3807	firedrake.functionspaceimpl.FunctionSpace._init_	9.6321
3	MatLUFactorNum	1.2108	MatLUFactorSym	2.6016	firedrake.functionspaceimpl.FunctionSpaceData.get_shared_data	9.6287
4	MatLUFactorSym	1.2096	MatLUFactorNum	1.7790	firedrake.functionspaceimpl.FunctionSpaceData._init_	9.6285
5	KSPSolve	0.3308	SNESJacobianEval	0.5060	firedrake.mesh.MeshTopology._facets	6.1145
6	PCApply	0.3186	KSPSolve	0.4807	firedrake.variational_solver.NonlinearVariationalSolver.solve	5.6465
7	MatSolve	0.3185	PCApply	0.4419	firedrake._init_	1.8317
8	SNESetUp	0.0691	MatSolve	0.4419	firedrake.assemble.assemble	1.3502
9	DMCreateMat	0.0681	SNESFunctionEval	0.2621	firedrake.variational_solver.NonlinearVariationalSolver._init_	1.2753
10	SNESJacobianEval	0.0552	MatMult	0.0324	firedrake.function.Function.interpolate	1.1178

Table 4: Top-10 timing events from `-log_view`: PETSc C run, PETSc solver events within Firedrake, and Firedrake wrappers. I apologize for the small size.

Problem 3

I believe the biharmonic equations still are only Eq. (5) and (6) in the problem statement for this question. We are prescribed a temperature gradient, so we don't need to solve Eq. (4) of this problem for temperature. Also the problem prescribes T with x, y but the code uses x, z so I am guessing the problem statement meant to use x, z and will continue with that assumption.

Fig. 1 shows a screenshot from Paraview of my modified solution.

Problem 4

Solving the full problem.

Part (a)

My code has been implemented in "convection_q4.py" in my fork of the course Github. It seems to run and kind of work! For this part, my final Nusselt Number was 0.99871. The velocity is below machine precision the whole simulation, so it is basically zero, which is expected for non-convective heat flow. The velocity about 35 seconds into this simulation is shown in Fig. 2.

Part (b)

Raleigh Number	Nusselt Number
10^2	0.99871
10^4	1.00007
10^5	1.00005
10^6	1.00532

It is odd that the Nusselt Number is not moving far from one. It should be increasing away from one. Something that I realized that is contradictory is that the homework assignment says to start with the initial temperature (and resulting ω, ψ) condition from Problem 3 of this homework. However, this

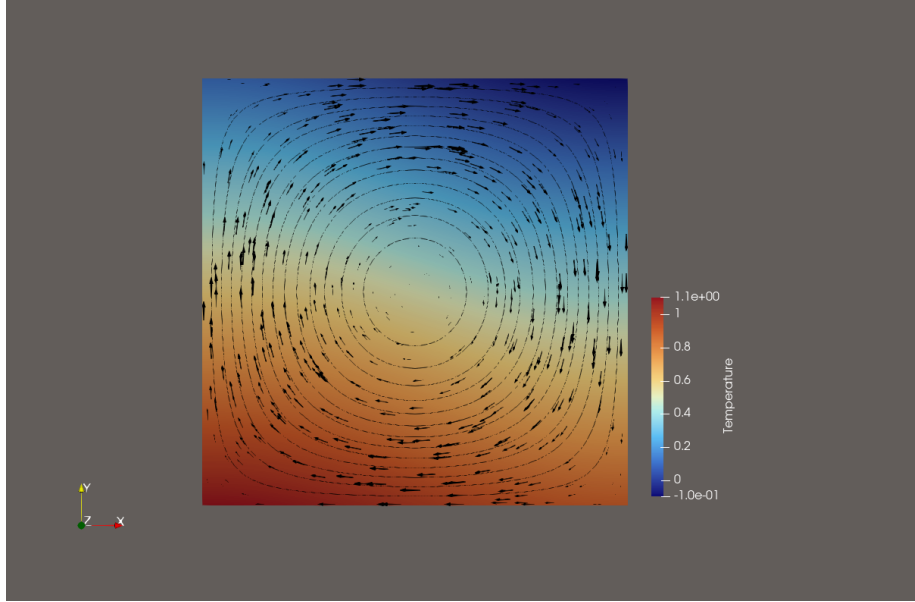


Figure 1: Temperature shown by heatmap plot. ψ streamline functions are shown in black. $\mathbf{v}$ arrows scaled based on vector magnitudes are also shown in black.

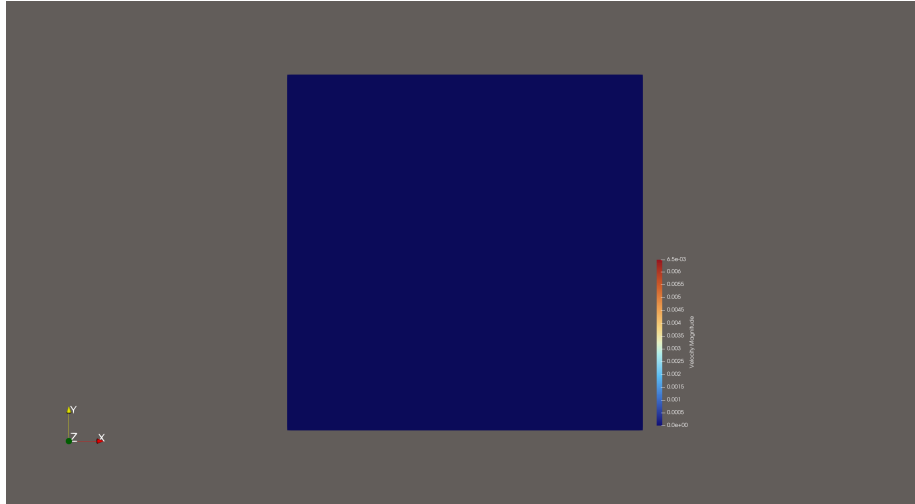


Figure 2: Velocity profile around 35 seconds of true time for Part (a) of Problem 4.

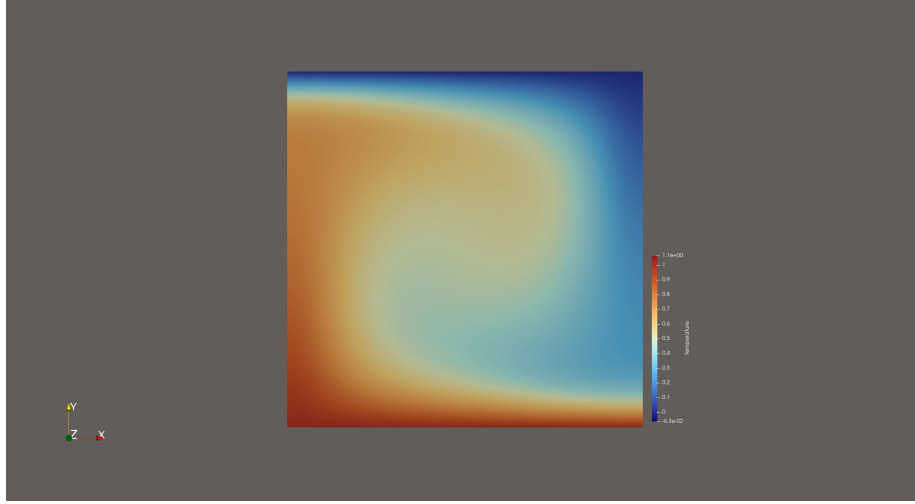


Figure 3: Temperature profile at final timestep for $Ra = 10^4$ with 64×64 mesh points.

initial condition does not satisfy the requested Dirichlet boundary conditions by problem 4. This is likely causing some issues and unphysical effects.

Sorry that I cannot get the right simulation going. I will continue to do Part (c) to have the code but there seems to be something off with these simulations so I don't think that part will be right either.

Part (c)

Mesh Size (N for $N \times N$)	Nusselt Number
16	1.00019
32	1.00008
64	1.00006
128	1.00003

Something is clearly wrong as these Nusselt numbers do not come close to the Blankenbach benchmark 1. I still think what is going wrong has to do with the boundary condition contradiction discussed in Part (b) of this problem. Possibly there is another issue with my code. It is interesting because the final temperature profile looks relatively normal, as shown in Fig. 3, but the velocity profile is basically always zero and the Nusselt numbers are all very close to one.